

CXU Playground

RISC-V Custom Extension Development Framework

Petro Mozil, supervised by Oleg Farenjuk

Ukrainian Catholic University

2026

What is RISC-V?

An open instruction set architecture

- ▶ RISC-V is a free and open ISA based on RISC principles [[@riscv-spec](#)]
- ▶ Unlike ARM or x86, anyone can implement it without licensing fees
- ▶ Maintained by the RISC-V International non-profit

Why it matters

- ▶ Modular: a small mandatory base + optional standard extensions (M, F, V, ...)
- ▶ Custom extension space (opcodes custom-0 through custom-3) is reserved for exactly this kind of work
- ▶ Growing ecosystem: Linux, GCC, LLVM, QEMU all support it

What is a CFU / CXU?

CFU — Custom Function Unit

- ▶ A hardware block attached to a RISC-V core that executes custom instructions
- ▶ Receives two register inputs (rs1, rs2), returns one result (rd)
- ▶ Defined by the CFU Logical Interface spec; reusable across cores
- ▶ **CFU Playground** provides a ready-made sandbox for developing CFUs

One extension per thread; invoked via inline assembly

CXU — Custom Extension Unit

- ▶ Next-generation spec: supports **multiple simultaneous extensions** selectable at runtime
- ▶ Shared persistent state via cxdata CSR
- ▶ C-level ABI — no inline assembly required
- ▶ Designed for cross-core portability

This work is (to the extent of the author's knowledge) the **first hardware implementation** of the CXU spec

Motivation – why use extension plugins at all?

RISC-V extensions

- ▶ RISC-V is designed with extensions in mind
- ▶ The instruction set allows for many custom commands
- ▶ Official extensions are managed by the RISC-V committee [riscv-spec]
- ▶ These extensions are hardly trivial to design – a lot must be kept in mind (memory ordering, compute constraints etc.)
- ▶ Extensions may share opcodes – this breaks compatibility between some extensions
- ▶ Extensions are hard to publish – it takes a while to ratify the extension

Motivation – why CXU?

CFU - exists

- ▶ Reusable across RISC-V cores
- ▶ Up to 255 functions
- ▶ CFU Playground: ready sandbox

One extension per thread,
assembly-only

CXU — spec only

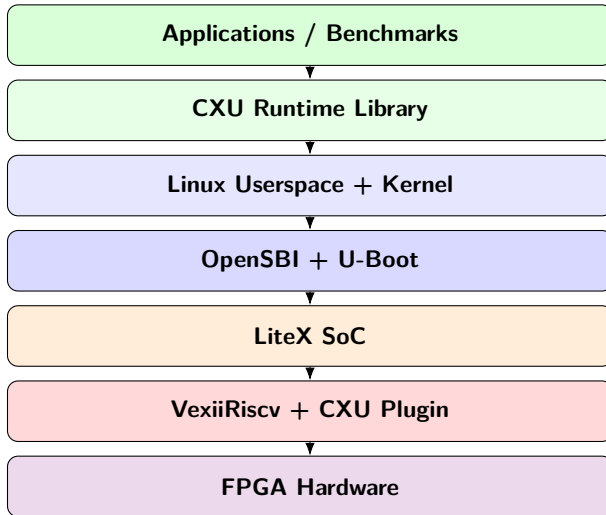
- ▶ Multiple simultaneous extensions
- ▶ Shared state
- ▶ C-level ABI, no inline assembly

Spec only for the extension, no
integration yet

This work

- ▶ First CXU hardware
- ▶ Full-stack open-source sandbox
- ▶ Benchmarked under Linux

Architecture



SpinalHDL

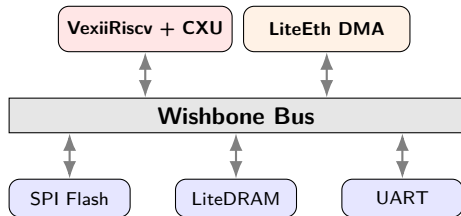
What it is

- ▶ Hardware description language built on Scala [`@spinalhdl`]
- ▶ Generates Verilog/VHDL
- ▶ Modular plugin system: no plugin may alter the behaviour of another one

CXU plugin enable

```
VexiiRiscvLitex(  
    plugins += CxuPlugin(...args)  
)
```

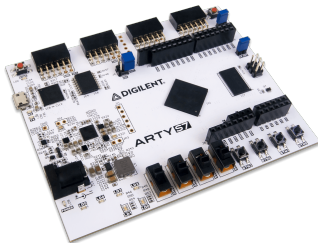
LiteX SoC Framework (enjoy-digital 2025)



Target hardware

Target: Arty S7-50

- ▶ The **Arty S7-50** is a development board by Digilent, designed for FPGA prototyping
- ▶ Carries a Spartan-7 XC7S50 FPGA
- ▶ 50 MHz, 256 MB DDR3



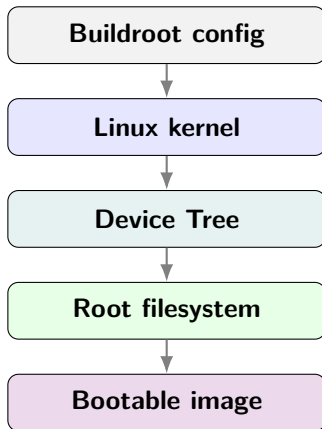
Arty S7-50 development board

FPGA utilisation (With AES example)

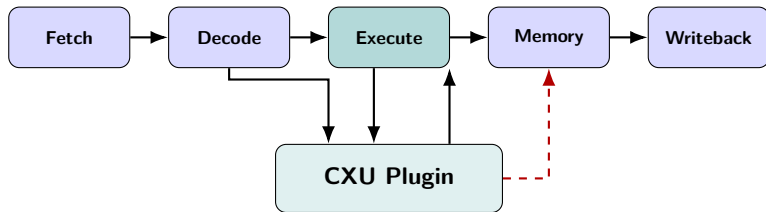
- ▶ CPU + SoC: 23% ($\approx$ 21,000 logic cells)
- ▶ Single CXU extension: $\approx$ 1 % (200 cells)
- ▶ Rest: free

Area figures tell the developer how much fabric remains — they are not a quality metric of the chip itself.

Buildroot (Mozil 2026a)

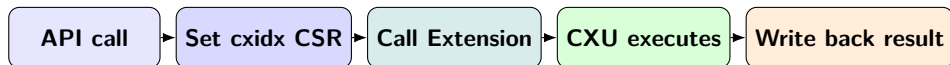


CXU Plugin



`cxidx` CSR selects active extension. Plugin stalls pipeline for multi-cycle ops.

CXU Instruction Execution Flow



CXU Runtime Library

Without the library

```
// Manual extension select
csrw cxidx, 2

// Custom opcode call (most compilers don't
support all RISC-V extensions)
.insn r CUSTOM_0, 0, 0,
    a0, a1, a2

// Core-specific, not portable
```

With the runtime library

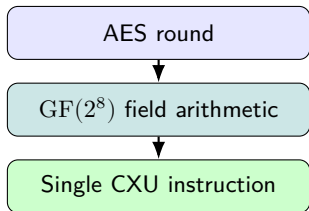
```
// Pure C --- no assembly
cxu_select(CXU_AES);
uint32_t r = cxu_call(a, b);

// Portable across CXU cores
```

Benchmark Methodology

- ▶ **Bare-metal:** `mcycle` / `minstret` counters
- ▶ **Linux:** timing with utils provided by POSIX time utils
- ▶ Multiple input sizes to separate fixed overhead from per-byte savings
- ▶ Compiler flags identical (force only `-O3`) for both baseline and CXU versions

Experiment: AES GF(2⁸) (Marshall et al. 2020)

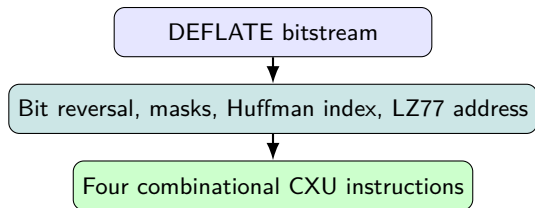


Experiment conditions

- ▶ Conducted **10 times** per input size on bare-metal and under Linux
- ▶ Compiler flags: -O3 only, identical for baseline and CXU versions
- ▶ Bare-metal timing: `mcycle` CSR snapshots (nearly deterministic)
- ▶ Linux timing: `clock_gettime`, variance $\leq \pm 2\%$

5.25 $\times$ speedup

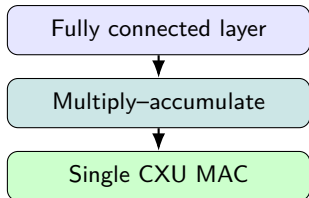
Experiment: GZIP / DEFLATE (Mozilla 2026b)



10% end-to-end speedup

{Experiment conducted **10 times** per file size; compiler flags -O3 for both versions. End-to-end gain is diluted by filesystem I/O at small file sizes (< 10 KB).}

Experiment: TFLite Micro MAC (David et al. 2021)



Experiment conditions

- ▶ Conducted **10 times** per layer size
- ▶ Compiler flags: -O3, identical for baseline and CXU
- ▶ **TFLite Micro does not use platform-specific SIMD/DSP optimizations** in this configuration — the baseline is purely scalar C

13% compute speedup

FPGA Resource Usage (Arty S7-50, XC7S50)

Extension	Logic cells	CXU cost	DSP48E1	Block RAM
AES	20,948	209 (1%)	4	36 tiles
GZIP	20,666	20	4	36 tiles
TFLite	20,698	folded	4	36 tiles
All three	21,394	229	4	36 tiles

A single CXU extension will likely (unless it's very complex) consume little FPGA fabric (\$ \$1% or less).

{**Note on area:** Logic cell counts are not a measure of chip quality. They are a practical indicator for the developer — showing how much fabric remains available for additional extensions or SoC peripherals.}

How to Add a New Extension

- ▶ Create the extension with the interface provided in examples (do mind - the names of the module go as 'Cxu0', 'Cxu1', and so on)
- ▶ Compile the system-on-chip with the created extension (add a cli argument: '-cxu <path-to-cxu-source>' to the build command)
- ▶ **Note - there's no Device Tree entries:** extension addresses hard-coded.

Limitations

- ▶ **CSR access latency:** `cxidx/cxdata` are Wishbone-mapped, not pipeline registers.
- ▶ **No context-switch support:** CXU state not saved on Linux task preemption — single-process only.
- ▶ **No Device Tree entries:** extension addresses hard-coded.
- ▶ **Clock frequency:** combinational paths reduce $F_{\max}$.

Future Work

- ▶ Process context-switch support (save/restore CXU state)
- ▶ Native pipeline CSRs (bypass Wishbone)
- ▶ Device Tree integration (dynamic discovery)
- ▶ Multi-process tests and benchmarks on Linux

Reviewer Questions: Benchmark Repeatability & Variance

Methodology

- ▶ Each benchmark run repeated **10 times** per input size
- ▶ Linux results: measured with `clock_gettime`
- ▶ Bare-metal results: `mcycle` / `minstret` CSR snapshots
- ▶ Variance was $\leq \pm 2\%$ for AES across all buffer sizes

Observed jitter sources

- ▶ Linux scheduler preemption ($\pm 5 - 6\%$ jitter, no effect on mean speedup)
- ▶ DDR3 refresh cycles
- ▶ GZIP: filesystem I/O dominates variance at small file sizes

Bare-metal runs are nearly deterministic – no measurable variance between runs.

Reviewer Questions: Quantifying Overhead Sources

Overhead Source	Cost	Workload Impact
Extension switch (cxidx change)	$\approx 50\text{--}60$ cycles	Amortized: batch calls per extension
Linux syscall / scheduler	$\pm 5\text{--}6\%$ jitter	No change to mean speedup
Filesystem I/O (GZIP)	Dominates at < 10 KB	Dilutes compute speedup end-to-end

Key takeaway

CXU switch latency is a **one-time configuration cost** – all three benchmarks batch extension calls to amortize it. The raw compute speedup (AES $5.25\times$) is unaffected; GZIP end-to-end gain ($\approx 10\%$) is diluted by I/O, not CSR overhead.

Reviewer Questions: Kernel Context-Switch Support

What needs to be done

- ▶ Add `cxidx + cxdata` to the Linux `task_struct`
- ▶ Instrument `__switch_to()` to save/restore those registers
- ▶ Add a kernel driver for mutual exclusion (replace current UIO shim)
- ▶ Expose extensions via `/dev/cxu` with `ioctl1` for process isolation

Estimated effort

- ▶ Context-switch patch: $\approx$ **1–2 weeks** (well-defined kernel hook)
- ▶ Kernel driver + locking: $\approx$ **2–4 weeks**
- ▶ Device Tree nodes: $\approx$ **1 week** (LiteX generator change)

CSR latency is a **LiteX integration issue**: mapping via Wishbone instead of native pipeline registers. Moving `cxidx/cxdata` to pipeline registers would bring latency to 2-3 cycles.

Reviewer Questions: Scalability to Memory-Accessing Extensions

Current constraint

- ▶ CXU instructions pass only rs1, rs2 → rd
- ▶ No direct bus-master port for the CXU – all data must flow through CPU registers
- ▶ TFLite MAC result ($\approx 13\%$) reflects this: most cycles are memory loads, not compute
- ▶ CXU spec also allows for a bigger state (*ge* 4KB). However, this implementation suffers from CSR latency as well.

Path to memory access

- ▶ CXU spec allows extensions to be granted a **memory port**
- ▶ Enables DMA-style bulk operations (e.g. full AES block in one call)
- ▶ Expected: order-of-magnitude further gains for memory-bound workloads

Summary

- ▶ CXU hardware implementation (SpinalHDL plugin for VexiiRiscv)
- ▶ CXU Playground: open-source, full-stack (LiteX + Buildroot + runtime lib)
- ▶ Three extensions that offer significant speedup: AES ($5.25\times$), GZIP (10%), TFLite MAC (13%)

<https://github.com/pmozil/CXU-playground>

<https://github.com/pmozil/vexiiriscv>

<https://github.com/pmozil/cxu-buildroot>

Similar extension interfaces / playgrounds

Feature	CFU Playground(Prakash et al. 2022)	FRANCIS-V(Egert 2023)	CXU Playground (this work)
Interface	CFU (single extension)	Core-V-XIF	CXU (multiple extensions)
Invocation	Assembly (custom-0/1/2/3)	C with compiler support	Pure C, portable API
Shared state	No	Yes (custom CSRs)	Yes (cxdata CSR)
OS support	Bare-metal only	Bare-metal + RTOS	Full Linux (Buildroot)
Portability	Across cores	CPU-specific	One core for now, designed for cross-core reuse as CFU playground

References

- David, Robert, Jared Duke, Advait Jain, Vijay Janapa Reddi, Nat Jeffries, Jian Li, Nick Kreeger, et al. 2021. "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems." In *Proceedings of Machine Learning and Systems*, 3:800–811. <https://arxiv.org/pdf/2010.08678>.
- Egert, Florian. 2023. "FRANCIS-v: Framework for Integrating Custom Instructions into RISC-v Systems." Master's thesis, TU Wien. <https://repositum.tuwien.at/bitstream/20.500.12708/188868/1/Egert%20Florian%20-%202023%20-%20FRANCIS-V%20FRAMework%20for%20iNtegrating%20Custom%20InstructionS...pdf>.
- enjoy-digital. 2025. "LiteX: Build Your Hardware, Easily!" <https://github.com/enjoy-digital/litex>.
- Marshall, Ben, G. Richard Newell, Dan Page, Markku-Juhani O. Saarinen, and Claire Wolf. 2020. "The Design of Scalar AES Instruction Set Extensions for RISC-V." *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2021 (1): 109–36. <https://doi.org/10.46586/tches.v2021.i1.109-136>.
- Mozil, Petro. 2026a. "Cxu-Buildroot: Buildroot Support for CXU-Enabled Systems." <https://github.com/pmozil/cxu-buildroot>; GitHub.
- . 2026b. "Gzip: Sample GZIP Compression Implementation for Accelerator Studies." <https://github.com/pmozil/gzip>; GitHub.
- Prakash, Shvetank, Tim Callahan, Joseph Bushagour, Colby Banbury, Alan V. Green, Pete Warden, Tim Ansell, and Vijay Janapa Reddi. 2022. "CFU Playground: Full-Stack Open-Source Framework for Tiny Machine Learning (tinyML) Acceleration on FPGAs." *arXiv Preprint*. <https://arxiv.org/abs/2201.01863>.